# 데이터 분석 실무 적용

Mark Young

# 실질적 활용에서 중요한 점

**모델 운영 연결 전략**:

- 모델 성능 검증 → 실시간/배치 적용 → 조직 협업 체계

**성과 측정**:

- Precision만이 전부가 아니다 → 실제 개선 KPI 설계법

**자동화**:

- 이탈 고객 자동 마케팅, 탐지 → 차단 시스템 연계

“실무에서는 좋은 모델보다 **안정적 운영**이 더 중요하다.”

# 금융 시스템

# 금융 시스템

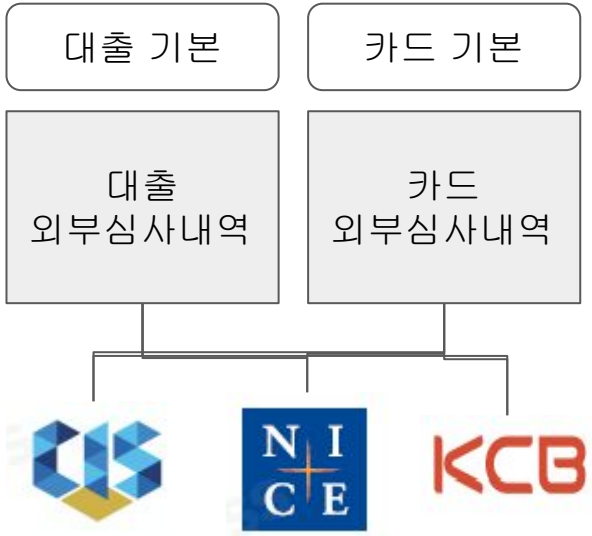
금융은 기본적으로 규제 산업이다.

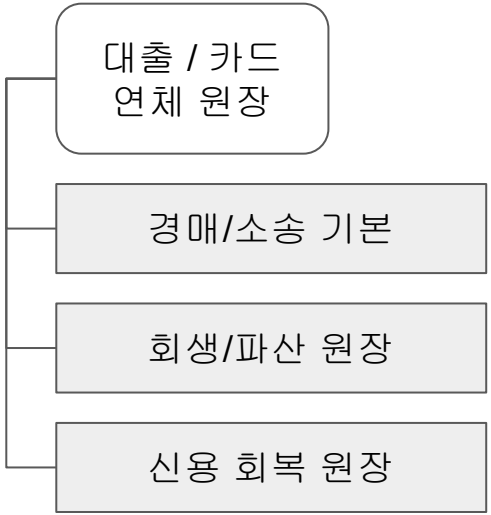
금융 업무는 업무별 기준 준칙이 있고, 제약사항과 법률적 기준을 준수해야 한다.

그렇기 때문에 DB Table을 유기적으로 엮어 놓으면 관리가 용의하지 않고, 규제나 법률 사항 변경 시 즉시 적용이 불가하다.

그래서 업무의 성격이나 프로세스에 따라 테이블의 통합과 분리가 고려되어야 하고 업무시스템 장애 방지를 위해서 느슨한 연결을 해 놓거나, 단일 DB로 아키텍처를 수립할 수 밖에 없다.

# 테이블의 분리와 통합




심사는 평가항목이 대출/카드가 상이한 항목이 많다
테이블을 심사내역 하나로 통합했을 때 불리한 점이 많다.

연체 이후는 대출이나 카드나 연체금액/연체일 등 공통적인 항목이 많다.
또 이후에 발생하는 법적절차 테이블들은 고객별 Data이다.

# 금융에서 현재 잘 관리되고 있는 테이블 목록

| 상품 | 금리 | 수수료 |
| --- | --- | --- |
| 여신 | 여신 | 여신 |
| 수신 | 수신 | 수신 |
| 카드 | 카드 | 카드 |

상품, 금리, 수수료가 각 팩토리에서 통합되어 관리됨

| 단일성 코드 | 목록성 코드 | |
| --- | --- | --- |
| Meta System | 공통코드 | 여신코드 |
| | 회계코드 | 수신코드 |
| | | 카드코드 |

고객 정보는 고객에 통합되어 관리되고 있어, 개인정보가 효율적으로 관리되고 있다.

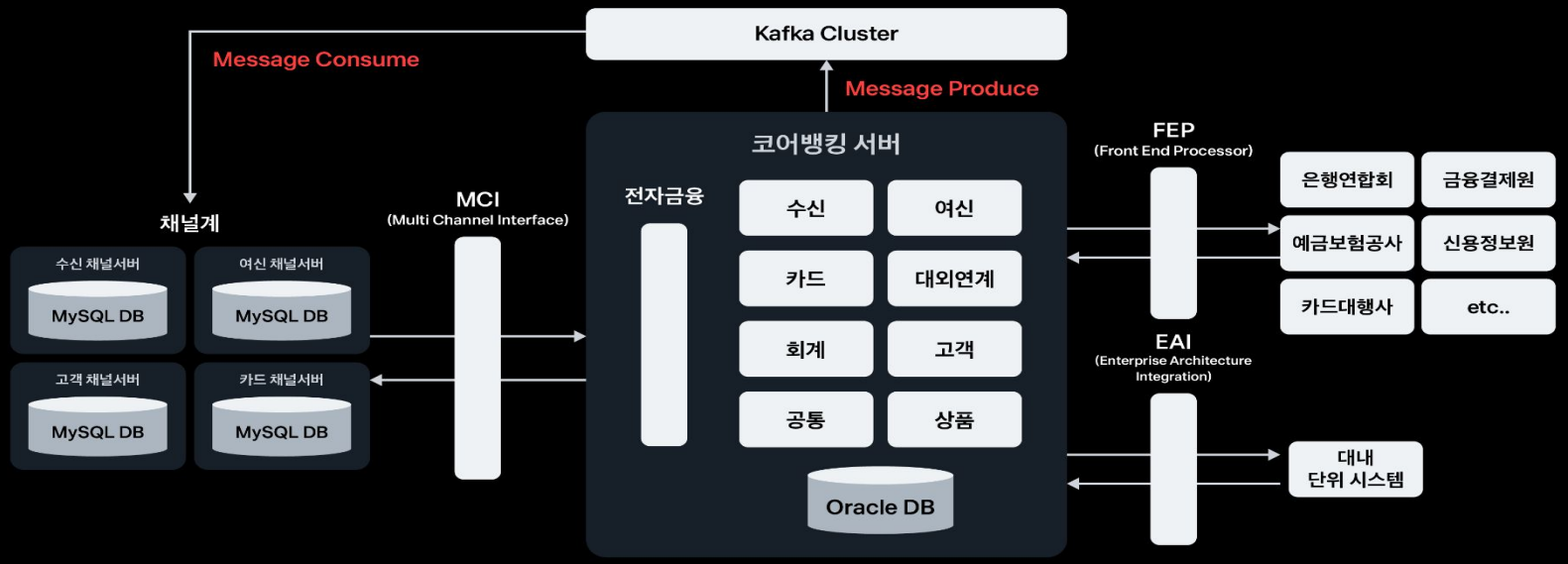
위와 같이 데이터 관리의 정답은 되도록 공통분모를 모아서 테이블을 설계하자.

데이터량/Transaction의 유형을 고려하여 공통분모를 모으자

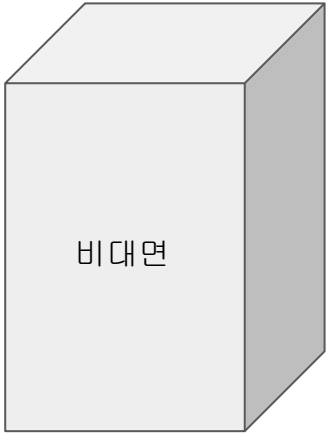
# 행 내 아키텍처 구성

SLASH 23

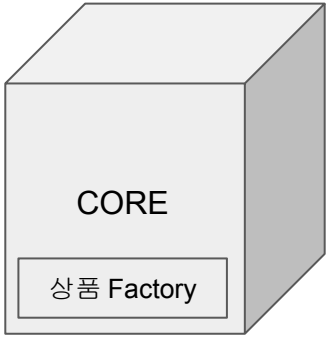
# 토스뱅크의 코어뱅킹 아키텍처




https://toss.tech/article/slash23-corebanking

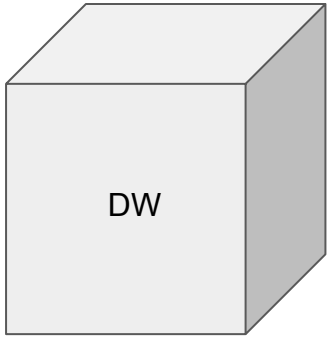



전자금융서버

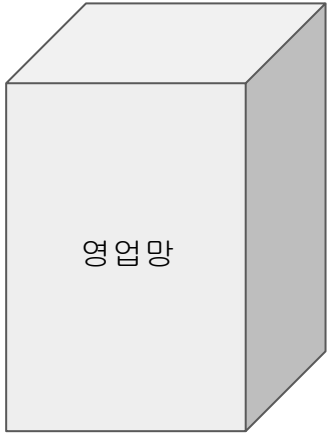



통합단말서버

| 은행 | 외화 | 골드 | 신탁 | 종금 |
|---|---|---|---|---|
| 외환법 ( 수수료 / 금리 / 상품 ) | | | 신탁법 | 종금법 |

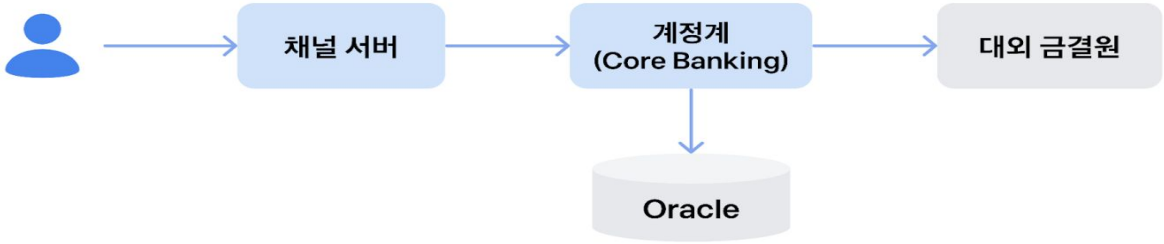



Data Warehouse

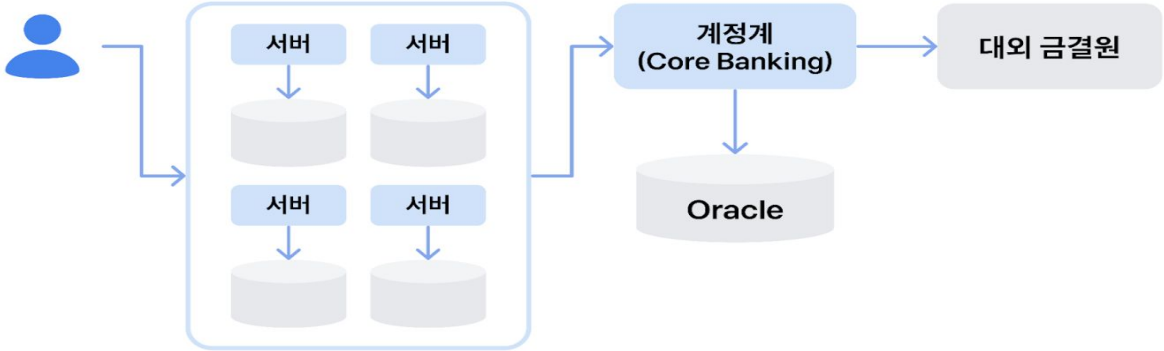



단위시스템

일반적인 은행의 시스템

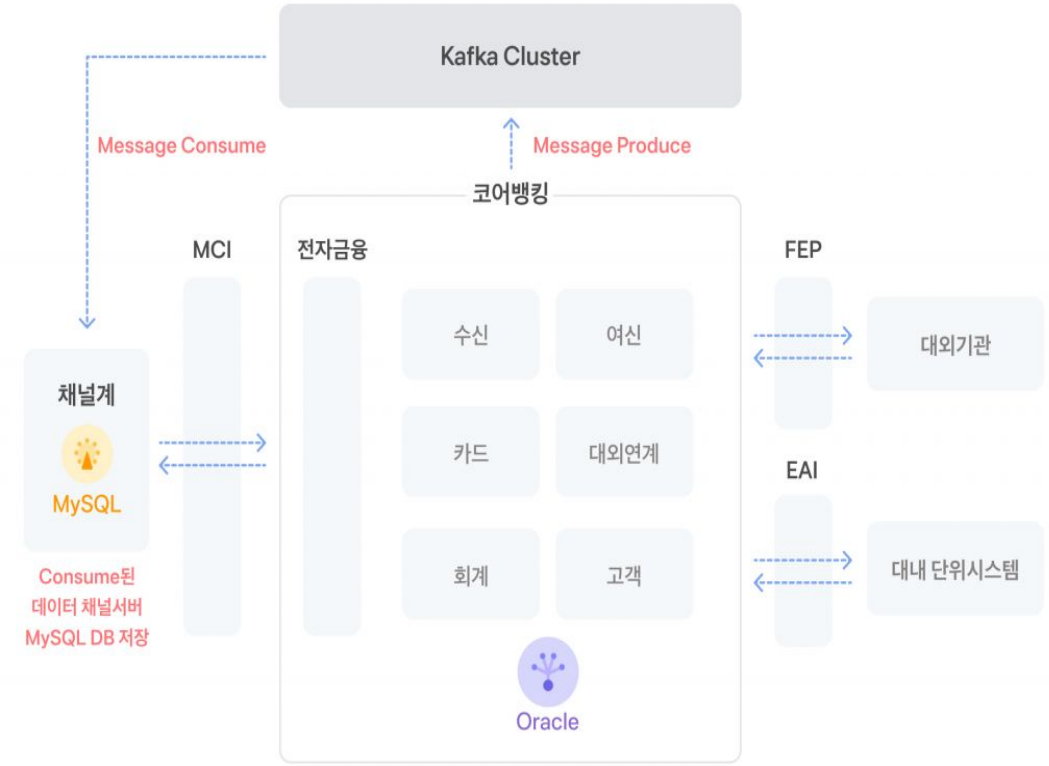



토스뱅크의 시스템




https://toss.im/career/article/tossbank-system

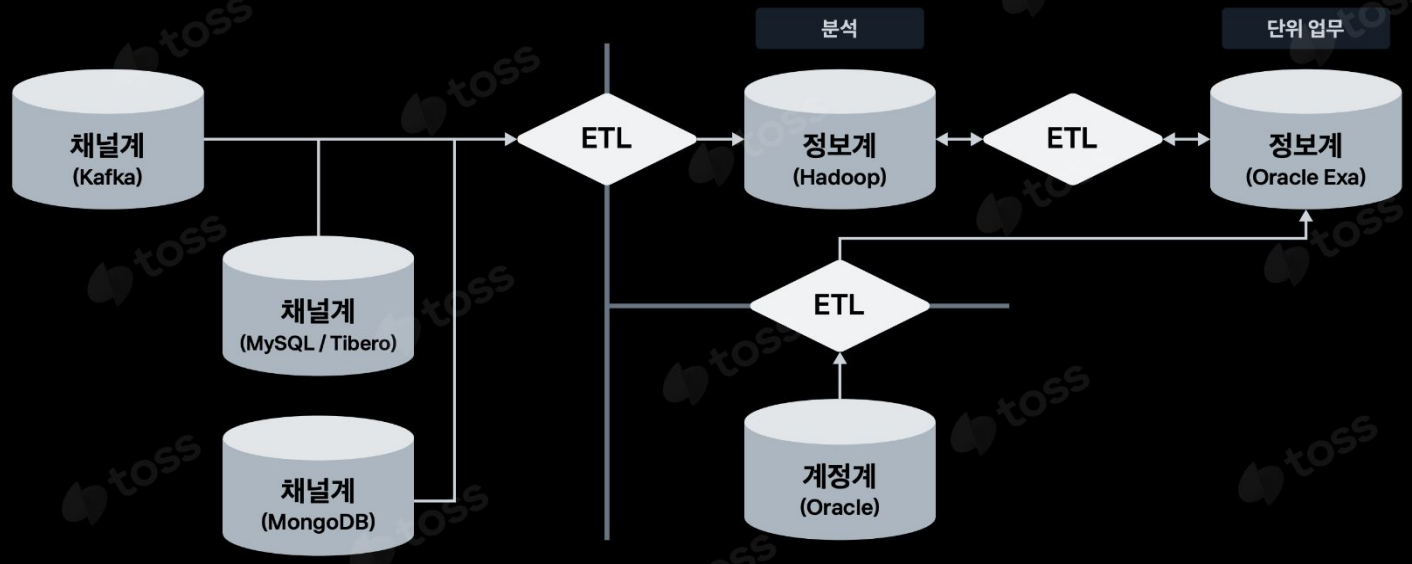
## 토스뱅크 코어뱅킹 아키텍쳐




https://blog.toss.im/article/tossbank-jobs-corebankingdeveloper

SLASH 23

# 일반적인 은행의 시스템 구조




https://www.youtube.com/watch?v=RjsG-zKMuX8

# LLMs 은행업 서비스 활용 방안

LLM 서비스 관점에서 보면 **은행이 막혀 있는 이유는 '기술 부족'이 아니라**
**LLM을 *프로세스 안에 어떻게 넣을지*를 잘 모르고 있기 때문**이야.

결론부터 말하면, 은행이 이 상황을 극복하는 방법은

> **LLM을 '의사결정자'로 쓰지 말고**
> **'프로세스 번역기·압축기·감시자'로 써야 한다.**

# 은행 프로세스의 착각: “LLM = 자동 판단”

## 은행이 본능적으로 거부하는 구조

- LLM이 승인/거절 판단
- LLM이 금리·한도 결정
- LLM이 고객 응대 단독 수행

이건 **법·감사·사고 리스크 때문에 절대 통과 안 됨**

## 대신 통과되는 구조

- **판단 전 단계 보조**
- **판단 후 검증**
- **판단 이력 정리**

은행에서 LLM은 **결정권이 없는 사람처럼 써야 한다.**

# Process를 쪼개면 LLM이 들어갈 틈이 보인다

## 전형적인 은행 업무 흐름

1. 데이터 수집
2. 규정 확인
3. 내부 기준 적용
4. 승인 판단
5. 보고·감사

**LLM은 2, 5번에서 가장 강력하다**

# 실전 적용 ① 규정·법률 “사람 해석 구간” 제거

## 현재 문제

- 내부 규정 PDF 수백 개
- 직원마다 해석 다름
- 민원/감사에서 터짐

## LLM 적용

- 법/내규/감독규정 → **벡터화**
- 질의:
  “이 고객은 고위험 투자자에 해당하나?”

- 출력:
  - 관련 조항
  - 과거 유사 사례
  - 해석 주의 포인트

중요한 포인트

- **판단** ❌
- **근거 제시** ⭕

LLM은 “결론”이 아니라 **판단 재료를 정리하는 도구**다.

# 실전 적용 ② 심사·상담 기록의 “사후 사고 방지”

**은행 사고의 70%**

- 설명의무 누락
- 기록 부실
- 말과 문서 불일치

**LLM 활용**

- 상담 음성 → 텍스트
- 필수 설명 키워드 체크
- 누락 시 실시간 알림
- 사후 감사용 요약 자동 생성

결과

- 사고가 줄어듦
- 감사 대응 시간 단축
- 직원 방어 논리 확보

은행은 **사고를 안 내는 조직이 아니라 사고가 나도 버틸 수 있는 조직**이다.

# 실전 적용 ③ 리스크 신호 “사람이 못 보는 패턴” 요약

## 기존 분석의 한계

- 수치 리포트
- 대시보드 과다
- 임원은 안 봄

## LLM의 역할

- 수백 지표 → **3줄 요약**
- 이상 원인 자연어 설명
- “이번 주 주의 지점” 자동 생성

예시 출력

“이번 주 ○○지점은
고령 고객 대상 고위험 상품 비중이
평소 대비 2.3배 증가했습니다.”

LLM은 **데이터 분석 결과를 임원이 이해할 수 있게 바꾸는 엔진**이다.

# Process 개선의 핵심: “LLM을 시스템 밖에 두지 마라”

## 실패하는 은행 LLM

- 챗봇 하나 띄움
- PoC에서 멈춤
- 현업과 분리

## 성공하는 구조

- **기존 승인 시스템 옆에 붙임**
- 모든 출력 로그화
- 결과는 “참고 의견”으로만 저장

아키텍처 감각

- LLM = Sidecar
- 원장/승인 시스템 = Core
- 로그 = 감사 자산

LLM은 **새 시스템이 아니라 기존 시스템의 보조 신경망**이다.

# 은행이 진짜 극복해야 할 건 기술이 아니다

## 진짜 장애물

- 책임 구조
- 사고 공포
- “내가 판단했다”는 흔적 남기기 싫음

## 그래서 **LLM**이 맞는 자리

- **누가 판단했는지 더 명확히 남기는 역할**
- 책임 전가가 아니라 **책임 명확화**

# LLMs & 데이터 분석

| 구분 | LLM | 데이터 분석가 |
| --- | --- | --- |
| 판단 | ❌ | ⭕ |
| 수치 계산 | ❌ | ⭕ |
| 설명/요약 | ⭕ | ❌ |
| 근거 연결 | ⭕ | ⭕ |
| 책임 | ❌ | ⭕ |

# 프로세스 단위 역할 분업 (실무 기준)

## ① 데이터 수집·정합성

- **분석가**
  - 원천 정의
  - 정합성 룰
  - 결측/이상 처리 기준
- **LLM**
  - 메타데이터 설명
  - 컬럼 의미 자연어 정리
  - “이 지표가 왜 필요한지” 문서화

결과

- 데이터 사전 자동 유지
- 신규 인력 온보딩 속도 ↑

## ② 규제·내규 해석

- **LLM**
  - 법/내규 벡터 검색
  - 관련 조항 요약
  - 해석상 주의점 제시
- **분석가**
  - 룰 수식화
  - 임계값 정의
  - 예외 조건 명시

핵심

LLM은 **조항을 말로**, 분석가는 **조건을 숫자로** 만든다.

## ③ 리스크 탐지 / 분석

- **분석가**
  - Feature 설계
  - Threshold 결정
  - 오탐/미탐 관리
- **LLM**
  - 결과 요약
  - 이상 원인 서술
  - 과거 유사 사례 연결

예시

- 분석가:

  “이 고객은 DSR 38%, 상환 패턴 급변”

- LLM:

  “최근 3개월 카드 사용 구조가
  소득 대비 급증한 것이 주요 원인”

# LLM 로그는 방패, 분석가 데이터는 증거

# Q & A

Thank you